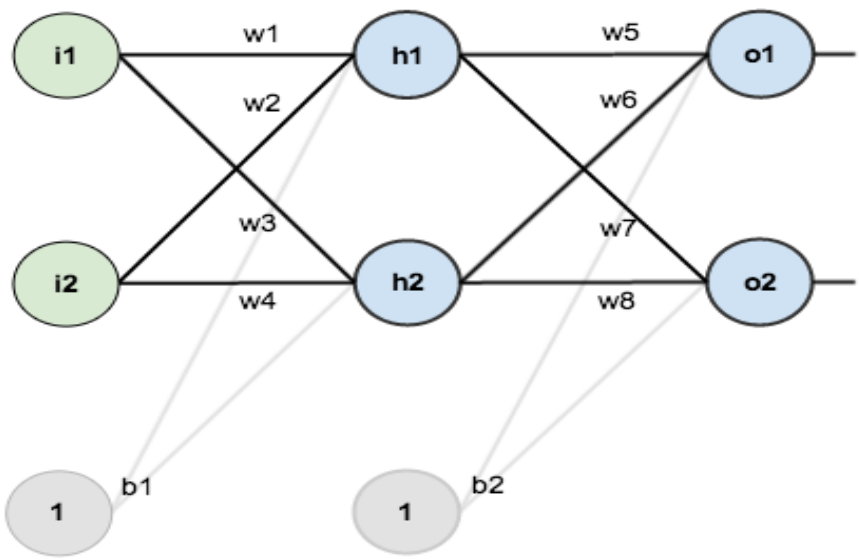
Back propagation Task

We're going to use a neural network with two inputs, two hidden neurons, and two output neurons. Additionally, the hidden and output neurons will include a bias.

Here's the basic structure:



X1	X2	Desired-1	Desired-2
0.05	0.10	0.01	0.99

W1	W2	W3	W4	W5	W6	W7	W8	B1	B2
0.15	0.20	0.25	0.30	0.40	0.45	0.50	0.55	0.35	0.6

The goal of back propagation is to optimize the weights so that the neural network can learn how to correctly map arbitrary inputs to outputs.

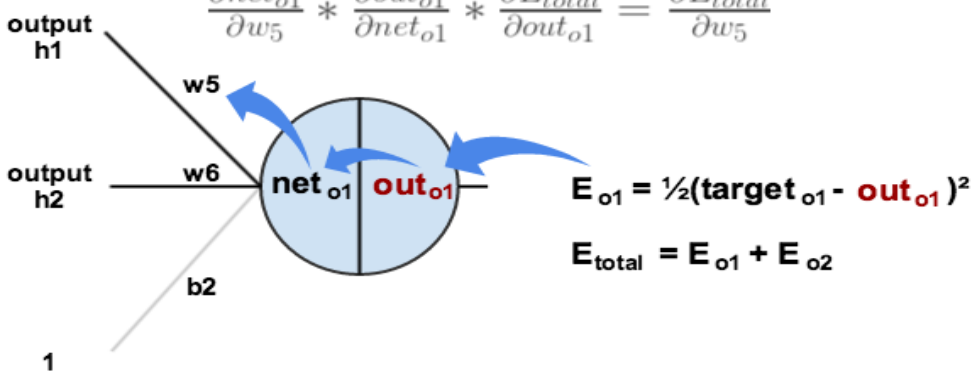
1- calculate the total net input for h_1
2- using the logistic function to get the output of h_1
3- Repeat step-1 and 2 to calculate h-2
4- Repeat this process for the output layer neurons, using the output from the hidden layer neurons as inputs.
5- <u>Calculating the Total Error using</u> $E_{total} = \sum \frac{1}{2}(target - output)^2$
6- <u>The Backwards Pass</u>
   With backpropagation, we want to minimize the error for each output neuron and the network overall by updating each weight in the network to bring the actual output closer to the intended output.

<u>Output Layer</u>

Consider w_5. We want to know how much a change in w_5 affects the total error,

By applying the chain rule

$$\frac{\partial net_{o1}}{\partial w_5} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial E_{total}}{\partial out_{o1}} = \frac{\partial E_{total}}{\partial w_5}$$



- **<u>how much does the total error change with respect to the output?</u>**

Note

> When we take the partial derivative of the total error with respect to out_{o1}, the quantity $\frac{1}{2}(target_{o2} - out_{o2})^2$ becomes zero because out_{o1} does not affect it which means we're taking the derivative of a constant which is zero.

- **<u>how much does the output of o_1 change with respect to its total net input?</u>**

Note

The partial c of the logistic function is the output multiplied by 1 minus the output

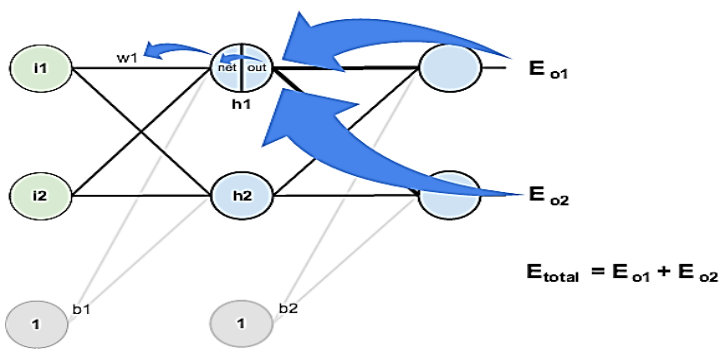
- **how much does the total net input of o1 change with respect to w_5?**
- **Repeat this process to get the new weights w_6, w_7, and w_8:**
-

Hidden Layer

- **calculating new values for w_1, w_2, w_3, and w_4.**

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



Finally, You've updated all of your weights! When you fed forward the 0.05 and 0.1 inputs originally

W1	W2	W3	W4	W5	W6	W7	W8	B1	B2

the error on the network was …….. After this first round of backpropagation, the total error is now down to …….. I think not seem like much.

Using the Bp.py Python code adjust the code look like this example to make sure your calculation for first iteration then repeating this process 10,000 times, when you feed forward 0.05 and 0.1, give me your prediction .